



École Mohammadia d'Ingénieurs

Département de Génie Informatique

Tests de Mutation

Mesurer la qualité réelle des tests unitaires d'EduStream

Réalisé par :
BAKLOUL Mohamed
HARBAL Salama

Encadré par :
Pr. RETBI

Année Universitaire : 2026 - 2027

Table des matières

1 Pourquoi des Tests de Mutation ? 3
1.1 Règle métier retenue . . . 3
1.2 Méthode suivie dans ce rapport . . . 3

2 Mise en Place du Moteur de Mutation 3
2.1 Fonctionnement du moteur . . . 4
2.2 Opérateurs de mutation . . . 5
2.3 Périmètre . . . 6

3 Tour 1 : Les Tests Unitaires Existants Face aux Mutants 6
3.1 Résultat global . . . 6
3.2 Limites révélées par les survivants . . . 6
3.3 Un mutant pour chaque limite . . . 7
3.4 Résultat de l'analyse . . . 8

4 Tour 2 : Écrire les Tests Tueurs 8
4.1 Premier lancement des tests tueurs : deux défauts réels . . . 9
4.2 Résultat du tour 2 . . . 10

5 Tour 3 : Derniers Survivants et Mutants Équivalents 11
5.1 Un vrai manque de test . . . 11
5.2 Trois mutants équivalents . . . 11
5.3 Score final . . . 12

6 Tests Ajoutés, Résultats et Couverture 12
6.1 Tests ajoutés . . . 12
6.2 Résultat de l'exécution . . . 12
6.3 Couverture . . . 13
6.4 Couverture et mutation ne mesurent pas la même chose . . . 14

7 Synthèse 14
7.1 Ce que le test de mutation a apporté en plus des tests unitaires . . . 14
7.2 Limites de la démarche . . . 15
7.3 Reproductibilité . . . 15

8 Conclusion 15

9 Benchmark : Stryker Mutator Face au Moteur Maison 16
9.1 Pourquoi comparer . . . 16
9.2 Mise en place . . . 16
9.3 Deux conceptions différentes . . . 17
9.4 Protocole du benchmark . . . 17
9.5 Résultats mesurés . . . 18
9.6 Tableau comparatif . . . 20
9.7 Conditions réelles de l'exécution de Stryker . . . 20
9.8 Démarche d'analyse des écarts . . . 21
9.9 Analyse des écarts . . . 21

9.10 Limites du benchmark 22

1 Pourquoi des Tests de Mutation ?

Le rapport précédent a montré que la suite Vitest passait avec une couverture de lignes de 75,81 %. Cette couverture mesure seulement les lignes **exécutées** pendant les tests, pas les comportements réellement **vérifiés**. Un test qui appelle une fonction sans contrôler son résultat fait monter la couverture sans protéger le code.

Le test de mutation répond à la question : *« si le code contenait une erreur, mes tests la détecteraient-ils ? »* Le principe est le suivant :

1. on introduit volontairement une petite erreur dans le code (un **mutant**), par exemple remplacer `<` par `<=`, supprimer une instruction ou vider un message d'erreur ;
2. on relance les tests unitaires ;
3. si au moins un test échoue, le mutant est **tué** (les tests ont détecté l'erreur) ; si tous les tests passent, le mutant **survit** : une partie du comportement n'est vérifiée par personne.

$$\text{score de mutation} = \frac{\text{mutants tués}}{\text{mutants tués} + \text{mutants survivants}}$$

1.1 Règle métier retenue

EduStream est un projet de test : **une carte qui passe la validation (algorithme de Luhn, date d'expiration, CVV, titulaire) est acceptée**. Aucun paiement valide ne doit être refusé par la simulation de paiement (ni au hasard, ni parce que la carte est « bloquée », ni à cause d'un solde simulé). Cette règle sert de référence pour écrire les tests tueurs.

1.2 Méthode suivie dans ce rapport

La démarche est la même que pour les tests unitaires : **écrire d'abord le dispositif de test, l'exécuter, relever les limites qu'il révèle, puis corriger**.

1. **Tour 1** : écrire le moteur de mutation et lancer la campagne sur les tests unitaires *existants* ; relever le score et chaque survivant.
2. **Analyse** : pour chaque survivant, déterminer s'il révèle un test manquant, un défaut du code, ou un mutant sans effet réel (*équivalent*).
3. **Tour 2** : écrire des tests « tueurs » ; les exécuter d'abord sur le code d'origine, et relever les échecs inattendus (défauts réels) ; corriger ; relancer la campagne.
4. **Tour 3** : traiter les derniers survivants, justifier les équivalents, valider le score final.

2 Mise en Place du Moteur de Mutation

Un ensemble de fichiers dédié a été créé à côté des tests unitaires. Il n'ajoute aucune dépendance : le moteur est écrit en JavaScript pour Node.js et pilote Vitest.

Il comprend :

— un mode d'emploi (`README`) ;

— le moteur (`run-mutation.mjs`) : il génère les mutants, lance les tests et calcule le score ;
— une configuration (`mutation.config.mjs`) : fichiers à muter et tests associés ;
— les opérateurs de mutation (`operators.mjs`) ;
— la liste des mutants équivalents avec leur justification (`equivalents.json`) ;
— les tests tueurs ajoutés pour éliminer les survivants (dossier `killers`) ;
— les résultats de chaque tour, au format JSON.

Deux commandes ont été ajoutées : `npm run test:mutation:baseline` (tour 1, tests existants seuls) et `npm run test:mutation` (tour 2, avec les tests tueurs).

2.1 Fonctionnement du moteur

— **Analyse du code** : chaque caractère est classé (code, chaîne, commentaire, expression régulière). Ainsi un `<` dans un commentaire ou un `+` dans une regex n'est jamais « muté » par erreur.
— **Un mutant à la fois** : le fichier est modifié, les tests concernés sont lancés avec `vitest run -bail=1` (arrêt au premier échec, donc rapide), puis le fichier est **toujours restauré** (bloc `finally`, signaux d'arrêt et fichier de sauvegarde `.mutation.bak`).
— **Test de référence** : avant la campagne, les tests doivent passer sur le code non modifié, sinon la campagne s'arrête.
— **Mutants invalides** : si TypeScript est installé, un mutant qui ne compile pas est écarté au lieu d'être compté comme « tué ».

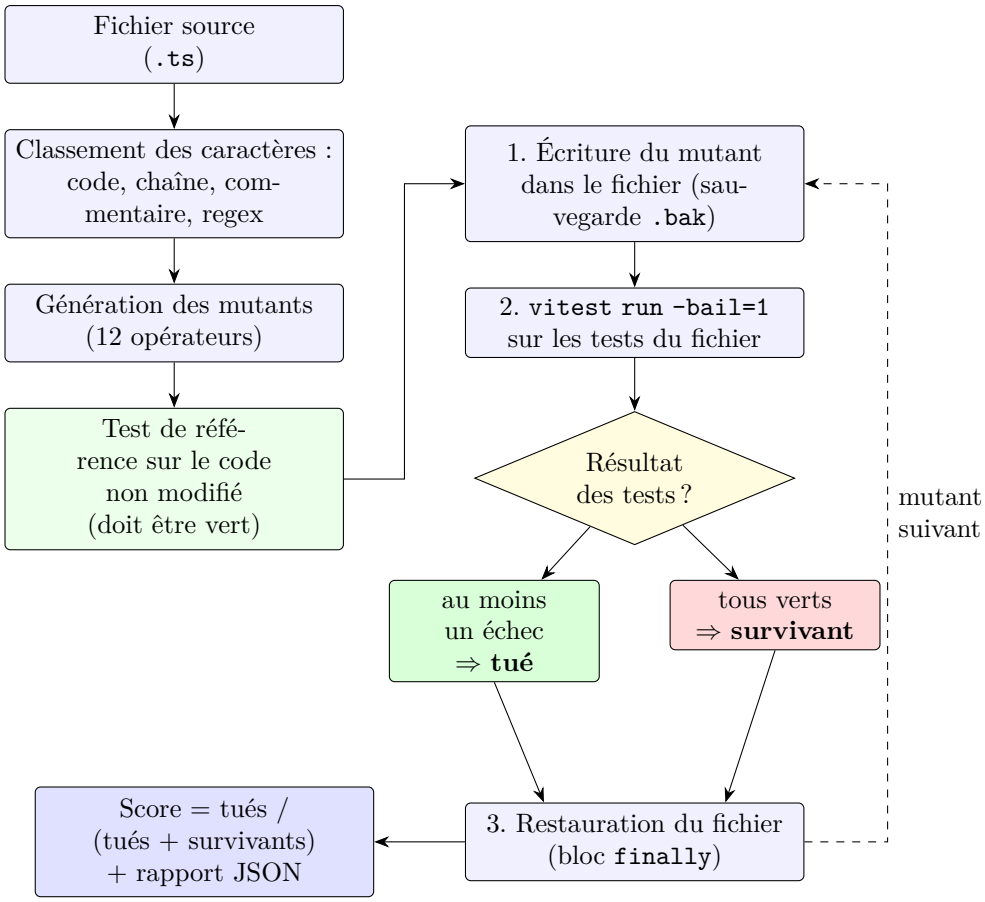


FIGURE 1 – Fonctionnement du moteur de mutation : un mutant à la fois, avec restauration systématique du fichier

2.2 Opérateurs de mutation

Douze opérateurs sont utilisés. Le tableau indique le nombre de mutants produits au tour 1.

Opérateur	Idée	Exemple	Mutants
`ConditionalBoundary`	Frontière de comparaison	`<` ↔ `<=`, `>` ↔ `>=`	14
`NegateConditional`	Égalité inversée	`===` ↔ `!==`	16
`LogicalOperator`	Opérateur logique	`&&` ↔ `||`	11
`ArithmeticOperator`	Opérateur arithmétique	`+ - * / %`	9
`BooleanLiteral`	Booléen inversé	`true` ↔ `false`	12
`IfCondition`	Condition forcée	`if (x)` → `if (true)` / `if (false)`	86
`NegationRemoval`	Négation supprimée	`!x` → `x`	21
`StatementRemoval`	Instruction supprimée	appel de méthode ou `throw` retiré	28
`StringLiteral`	Chaîne vidée	`'failed'` → `""`	23
`NumberLiteral`	Nombre modifié	$n \to n+1$	25
`RegexLiteral`	Expression régulière	ancre `^` retirée, `\d` ↔ `\D`, `+` → `*`	19
`MethodExpression`	Méthode	`Math.max` ↔ `Math.min`, `.trim()` retiré	3

TABLE 1 – Opérateurs de mutation et nombre de mutants générés (tour 1)

2.3 Périmètre

La campagne porte sur six fichiers (267 mutants au tour 1) : `validators.ts`, `payment.service.ts`, `enrollment.service.ts`, `student.service.ts`, `course.service.ts` et `implementation.ts`. Seuls les tests **unitaires** rapides sont utilisés (`validators.test.ts` et `services.test.ts`) : les tests d'intégration Next.js et Selenium n'en font pas partie.

3 Tour 1 : Les Tests Unitaires Existants Face aux Mutants

3.1 Résultat global

Le test de référence passe (81 tests sur le code non modifié). La campagne produit 267 mutants : **180 sont tués et 87 survivent**, soit un score de **67,4 %**.

```
PS C:\Users\YOGA\Desktop\emi\s5\securite_processus_gl\architecture-application-edustream-master\architecture-application-edustream-master>n
pm run test:mutation:baseline
[245/255] TUÉ       lib/repositories/implementation.ts:142  StatementRemoval     "this.payments.set(payment.id, payment)" → ";"
[246/255] SURVIVANT lib/repositories/implementation.ts:147  LogicalOperator      "||" → "&&"
[247/255] SURVIVANT lib/repositories/implementation.ts:152  IfCondition          "payment.enrollmentId === enrollmentId" → "true"
[248/255] SURVIVANT lib/repositories/implementation.ts:152  IfCondition          "payment.enrollmentId === enrollmentId" → "false"
[249/255] SURVIVANT lib/repositories/implementation.ts:152  NegateConditional    "===" → "!=="
[250/255] SURVIVANT lib/repositories/implementation.ts:159  NegationRemoval      "!" → ""
[251/255] SURVIVANT lib/repositories/implementation.ts:159  IfCondition          "!existing" → "true"
[252/255] SURVIVANT lib/repositories/implementation.ts:159  IfCondition          "!existing" → "false"
[253/255] SURVIVANT lib/repositories/implementation.ts:159  StringLiteral        "'Payment not found'" → "''"
[254/255] SURVIVANT lib/repositories/implementation.ts:161  StatementRemoval     "this.payments.set(id, updated)" → ";"
[255/255] TUÉ       lib/repositories/implementation.ts:167  NegateConditional    "===" → "!=="

=============== SYNTHÈSE ===============
Tour 1 — score de mutation : 70.2 %  (tués 179, survivants 76, équivalents 0, invalides 0)
  lib/services/validators.ts              95.5 %   (128 tués / 6 survivants / 0 équivalents)
  lib/services/payment.service.ts         22.2 %   (2 tués / 7 survivants / 0 équivalents)
  lib/services/enrollment.service.ts      72.0 %   (18 tués / 7 survivants / 0 équivalents)
  lib/services/student.service.ts         39.3 %   (11 tués / 17 survivants / 0 équivalents)
  lib/services/course.service.ts          16.7 %   (2 tués / 10 survivants / 0 équivalents)
  lib/repositories/implementation.ts      38.3 %   (18 tués / 29 survivants / 0 équivalents)
```

FIGURE 2 – Tour 1 : score de mutation par fichier avec les tests unitaires initiaux

Le score est très inégal. Les validateurs, pourtant testés à fond, atteignent 95,5 %, alors que `payment.service.ts` n'obtient que 8,0 % : il existe du code que les tests exécutent ou ignorent sans le vérifier réellement.

3.2 Limites révélées par les survivants

Limite 1 : la passerelle de paiement n'est vérifiée par aucun test unitaire (8,0 %)

`services.test.ts` remplace la passerelle par un objet simulé qui répond toujours « succès » ou « refus ». Le vrai `MockPaymentGateway` n'est donc jamais contrôlé. Dans le code d'origine il contenait en plus de la validation deux règles de refus : un **solde simulé** calculé à partir des quatre derniers chiffres de la carte et une **carte « bloquée »** (`4111 1111 1111 1111`). Les survivants le montrent : supprimer la règle des fonds insuffisants, modifier la formule du solde ou vider le motif « Carte bloquée » ne fait échouer aucun test. Ces survivants posent surtout une question : *ces règles sont-elles compatibles avec la règle « carte valide = carte acceptée » ?*

Limite 2 : la baisse du nombre de places n'est jamais vérifiée (`enrollment.service.ts`, 71,4 %)

Les survivants `await this.courseRepository.updateAvailableSeats(courseId, 1)` → `;` (instruction supprimée) et `1` → `2` révèlent un trou important : **aucun test ne contrôle qu'une inscription réussie retire une place**. Les tests de la section « épuisement des places (500 → 0) » vérifient seulement qu'un cours déjà à zéro refuse l'inscription et qu'un refus de paiement conserve les places. Si la ligne qui décrémente était supprimée par erreur, la suite resterait verte alors que la fonctionnalité serait cassée. De même, `<= 0` → `<= 1` survit : personne ne teste l'achat de la **dernière place**.

Limite 3 : authentification, doublons d'e-mail et mise à jour non testés (`student.service.ts`, 39,3 %)

Aucun test n'appelle `authenticateStudent`, ni le refus d'un e-mail déjà utilisé, ni le conflit d'e-mail lors d'une mise à jour. Les mutants `||` → `&&` dans la vérification des identifiants (qui accepterait un mauvais mot de passe) survivent : c'est la limite la plus sensible pour la sécurité.

Limite 4 : consultation des cours et dépôts en mémoire (`course.service.ts`, 16,7 % ; `implementation.ts`, 46,8 %)

Le filtre `availableSeats > 0` des cours disponibles, le calcul du nombre d'inscrits, les erreurs « introuvable » et la suppression ne sont pas vérifiés. Un mutant qui afficherait les cours complets comme disponibles passe inaperçu.

Limite 5 : détails des validateurs (95,5 %)

Les validateurs ont peu de survivants, mais ils montrent deux oublis : les sous-plages Mastercard `2230-2719` ne sont pas testées (seules 2221 et 2720 l'étaient), et l'ancre `^` de la détection AMEX n'est pas vérifiée (un numéro contenant « 34 » au milieu serait pris pour une AMEX).

3.3 Un mutant pour chaque limite

Pour comprendre ce que le moteur a réellement testé, voici un survivant représentatif de chacune des cinq limites : la ligne d'origine, le mutant produit, et la raison pour laquelle les tests existants restaient verts.

```
// Limite 1 - payment.service.ts (passerelle d'origine)
// original :
if (simulatedBalance < amount) {...}
// mutant (survit) :
if (simulatedBalance <= amount) {...}
// raison : le test remplace la passerelle par un objet simule

// Limite 2 - enrollment.service.ts
// original :
await this.courseRepository.updateAvailableSeats(courseId, 1)
// mutant (survit) : l'instruction disparait
;
```

```
// raison : aucun test ne verifie qu'une place est retiree

// Limite 3 - student.service.ts
// original :
if (!student || student.password !== password)
// mutant (survit) :
if (!student && student.password !== password)
// raison : authenticateStudent n'est appelee par aucun test

// Limite 4 - course.service.ts
// original :
return courses.filter((c) => c.availableSeats > 0)
// mutant (survit) :
return courses.filter((c) => c.availableSeats >= 0)
// raison : un cours complet serait affiche comme disponible

// Limite 5 - validators.ts
// original :
if (/^3[47]/.test(digits)) return 'AMEX'
// mutant (survit) :
if (/3[47]/.test(digits)) return 'AMEX'
// raison : aucun test avec 34 ou 37 au milieu du numero
```

Une fois le survivant connu, le test qui le tue est simple à écrire. Pour la limite 3, par exemple, il suffit de vérifier qu'un mauvais mot de passe est refusé.

3.4 Résultat de l'analyse

Chaque survivant a été rangé dans l'une de trois catégories : **test manquant** (à corriger par un test tueur), **défaut potentiel du code** (à confirmer par un test écrit sur le code d'origine), **équivalent** (sans effet réel, à justifier). Cette classification est faite au tour 2.

4 Tour 2 : Écrire les Tests Tueurs

Pour chaque limite, un fichier de tests a été ajouté dans le dossier `killers` (73 tests au tour 2, 74 après le tour 3 ; un fichier par module). La règle est la suivante : **le test tueur est d'abord exécuté sur le code d'origine, non muté**. S'il échoue, ce n'est plus un simple manque de test mais un **défaut réel du code**.

```
// Tue : await this.courseRepository.updateAvailableSeats(
   courseId, 1) -> ;
it('une␣inscription␣reussie␣retire␣EXACTEMENT␣une␣place␣(500␣->␣
   499)', async () => {
  const w = makeWorld()
  const student = await newStudent(w)
  const course = await newCourse(w, 1000, 500)
  await w.enrollmentService.enrollStudent(student.id, course.id,
     validCard())
  const after = await w.courseService.getCourseDetails(course.id)
  expect(after.availableSeats).toBe(499)
```

```
})

// Regle du projet : carte valide (Luhn) => acceptee, quel que
   soit le montant
it('accepte une carte valide quel que soit le montant', async ()
   => {
  for (const amount of [1, 16000, 16001, 49999, 50000, 1_000_000
     ]) {
    await expect(gateway.processPayment(amount, validCard()))
      .resolves.toMatch(/^TXN-/)
  }
})
```

4.1 Premier lancement des tests tueurs : deux défauts réels

Sur le code d'origine, **5 des 74 tests tueurs échouent**. Ces échecs ne correspondent à aucun mutant : ils révèlent des erreurs qui existaient déjà.

Défaut réel A : la passerelle refuse des cartes parfaitement valides

Le code d'origine contenait deux règles de refus qui contredisent la règle du projet :

```
const simulatedBalance = (lastDigits * 1000) % 50000
if (simulatedBalance < amount) {
  throw new InsufficientFundsException()      // refus selon les
     4 derniers chiffres
}
if (cardData.cardNumber.includes('4111111111111111')) {
  throw new PaymentRefusedException('Carte bloquee')   // carte
     de test refusee
}
```

Les tests tueurs écrits à partir de la règle « carte valide = acceptée » échouent sur le code d'origine :

- la carte `4111 1111 1111 1111`, valide au sens de Luhn, est **refusée** (« Carte bloquée ») ;
- une carte valide dont les quatre derniers chiffres sont `0000` a un solde simulé de zéro : **tout paiement est refusé** ;
- une carte valide est refusée dès que le montant dépasse un « solde » arbitraire (par exemple 16 001 DH pour une carte finissant par `0366`).

C'est un défaut du même type que le tirage aléatoire du rapport précédent : un refus qui ne dépend pas de la validité de la carte. Une première version des tests avait d'ailleurs montré que le blocage de la carte se contournait en la saisissant avec des espaces (`includes` sur le numéro brut) ; cela a conduit à se demander si cette règle devait exister.

Solution trouvée. Pour un projet de test, ces règles n'apportent que des refus imprévisibles. La passerelle simulée ne fait plus que valider la carte, puis renvoie un identifiant de transaction :

```
async processPayment(_amount: number, cardData: CardData):
   Promise<string> {
```

```
  CardValidator.validateCardData(cardData)     // Luhn, expiration
     , CVV, titulaire
  return `TXN-${uuid().substring(0, 8)}`
}
```

Les tests tueurs de la passerelle vérifient maintenant que **toute** carte valide est acceptée (avec ou sans espaces, tous montants, toutes terminaisons, 300 paiements d'affilée sans refus) et qu'une carte invalide est refusée avant tout paiement.

Défaut réel B : un paiement refusé interdit toute nouvelle tentative

Le contrôle anti-doublon cherchait *n'importe quelle* inscription du même étudiant au même cours. Or un paiement refusé laisse une inscription au statut `failed`. Le test « après un paiement refusé, l'étudiant peut réessayer » échoue avec :

```
EnrollmentAlreadyExistsException: L'etudiant 9f13048a-... est
deja inscrit au cours 6a0218e7-...
```

Conséquence métier : un étudiant dont la carte est refusée (mauvaise saisie, fonds insuffisants) ne peut plus jamais s'inscrire à ce cours, même avec une autre carte. Combiné avec le défaut « vibe coded » du rapport précédent (10 % de cartes refusées au hasard), **environ un étudiant sur dix était bloqué définitivement**. Même sans refus aléatoire, une simple erreur de saisie suffit à produire un refus (un échec de passerelle réel, par exemple).

Solution trouvée. Le deuxième test (« après une tentative refusée puis réussie, un troisième achat reste interdit ») a précisé la règle attendue : les inscriptions `failed` ne comptent pas, seules les inscriptions `pending` (paiement en cours, évite le double clic) et `completed` bloquent un doublon.

```
const studentEnrollments = await this.enrollmentRepository.
   findByStudent(studentId)
const alreadyEnrolled = studentEnrollments.some(
  (e) => e.courseId === courseId && e.status !== 'failed'
)
if (alreadyEnrolled) {
  throw new EnrollmentAlreadyExistsException(studentId, courseId)
}
```

Après correction, tous les tests tueurs et les 81 tests existants passent (voir section « Tests ajoutés, résultats et couverture »).

Code mort découvert

Le survivant `private minBalance = 100` → `101` (`payment.service.ts`, ligne 15) ne peut être tué par aucun test, car ce champ n'est utilisé nulle part. Il a été supprimé.

4.2 Résultat du tour 2

Le code corrigé a produit 4 survivants sur 255 mutants (le total passe de 267 à 255 car la passerelle est plus courte : les règles de solde et de carte bloquée ont disparu). Score : **98,4 %** (251 tués).

5 Tour 3 : Derniers Survivants et Mutants Équivalents

Les 4 survivants restants ont été analysés un par un.

5.1 Un vrai manque de test

`student.service.ts`, ligne 86 : `if (!student)` → `if (false)` dans `updateStudent`. Le test d'un étudiant inconnu utilisait une mise à jour sans e-mail : le code continuait jusqu'au dépôt, qui lève la même exception, d'où la survie. Avec un **nouvel e-mail**, le mutant provoquerait une `TypeError` en lisant `student.email` sur une valeur nulle. Un test a été ajouté :

```
it('etudiant inconnu avec un nouvel e-mail => StudentNotFoundException', async () => {
  await expect(w.studentService.updateStudent('inconnu', { email: 'nouveau@edustream.ma' }))
    .rejects.toThrow(StudentNotFoundException)
})
```

5.2 Trois mutants équivalents

Un mutant est **équivalent** lorsqu'il change le code sans jamais changer le comportement observable : aucun test ne peut le tuer. Chacun a été justifié et enregistré dans `equivalents.json`, de sorte qu'il ne soit plus compté.

Fichier	Opérateur	Pourquoi le comportement est identique
`validators.ts`	`RegexLiteral`	La longueur (13 à 19 chiffres) est vérifiée juste après : une chaîne vide ou faite uniquement de séparateurs contient 0 chiffre et est refusée de toute façon.
`validators.ts`	`ConditionalBoundary`	Un chiffre doublé est toujours pair : il ne peut jamais valoir exactement 9, donc `> 9` et `>= 9` donnent le même résultat.
`course.service.ts`	`IfCondition`	Garde redondante : si le cours n'existe pas, courseRepository.update lève exactement la même CourseNotFoundException.

TABLE 2 – Mutants équivalents justifiés

Pour le deuxième cas (`digit > 9` / `>= 9`), un chiffre doublé est forcément pair : il ne peut donc jamais valoir exactement 9. Le premier est protégé par la vérification de longueur qui suit (13 à 19 chiffres), et le troisième par une garde redondante : si le cours n'existe pas, le dépôt lève la même exception.

5.3 Score final

```
PS C:\Users\YOGA\Desktop\emi\s5\securite_processus_gl\architecture-application-edustream-master\architecture-application-edustream-master>n
pm run test:mutation
[266/271] TUÉ       lib/repositories/implementation.ts:159  NegationRemoval      "!" → ""
[267/271] TUÉ       lib/repositories/implementation.ts:159  IfCondition          "!existing" → "true"
[268/271] TUÉ       lib/repositories/implementation.ts:159  IfCondition          "!existing" → "false"
[269/271] TUÉ       lib/repositories/implementation.ts:159  StringLiteral        "'Payment not found'" → "''"
[270/271] TUÉ       lib/repositories/implementation.ts:161  StatementRemoval     "this.payments.set(id, updated)" → ";"
[271/271] TUÉ       lib/repositories/implementation.ts:167  NegateConditional    "===" → "!=="

=============== SYNTHÈSE ===============
Tour 2 — score de mutation : 100.0 %  (tués 267, survivants 0, équivalents 4, invalides 0)
  lib/services/validators.ts              100.0 %   (132 tués / 0 survivants / 2 équivalents)
  lib/services/payment.service.ts         100.0 %   (24 tués / 0 survivants / 1 équivalents)
  lib/services/enrollment.service.ts      100.0 %   (25 tués / 0 survivants / 0 équivalents)
  lib/services/student.service.ts         100.0 %   (28 tués / 0 survivants / 0 équivalents)
  lib/services/course.service.ts          100.0 %   (11 tués / 0 survivants / 1 équivalents)
  lib/repositories/implementation.ts      100.0 %   (47 tués / 0 survivants / 0 équivalents)

Rapport JSON : tests/mutation/reports/round-2.json
PS C:\Users\YOGA\Desktop\emi\s5\securite_processus_gl\architecture-application-edustream-master\architecture-application-edustream-master>
```

FIGURE 3 – Tour 3 : campagne finale, mutants équivalents ignorés

Sur 252 mutants valides, 252 sont tués et 0 survivent : score de mutation de **100,0 %**.

6 Tests Ajoutés, Résultats et Couverture

Cette section donne le bilan réel des tests écrits pendant la campagne : combien sont-ils, passent-ils, et que couvrent-ils ? Les chiffres proviennent d'une exécution de `npm run test:frontend:coverage` (Vitest 5.0.1, couverture V8) sur le poste du projet, une fois la campagne de mutation terminée.

6.1 Tests ajoutés

Fichier de test	Tests	Origine
`unit/validators.test.ts`	74	rapport précédent (améliorés)
`unit/services.test.ts`	7	existant
`unit/enrollment-client.test.ts`	8	existant
`integration/api-routes.test.ts`	5	existant
`components/ui-components.test.tsx`	4	existant
`mutation/killers/validators.killers.test.ts`	15	**ajouté** (tour 2)
`mutation/killers/payment.killers.test.ts`	13	**ajouté** (tour 2)
`mutation/killers/enrollment.killers.test.ts`	15	**ajouté** (tour 2)
`mutation/killers/student.killers.test.ts`	14	**ajouté** (tours 2 et 3)
`mutation/killers/course.killers.test.ts`	8	**ajouté** (tour 2)
`mutation/killers/repositories.killers.test.ts`	9	**ajouté** (tour 2)
Total	**172**	dont **74 tests tueurs**

TABLE 3 – Inventaire des tests (11 fichiers)

6.2 Résultat de l'exécution

Les **172 tests passent** (11 fichiers sur 11, aucun échec) en 15,96 s. L'exécution avec couverture produit d'abord un avertissement : Vitest 5.0.1 était associé à `@vitest/coverage-v8`

5.0.3, ce qui est une combinaison non supportée. Les deux paquets ont été alignés sur la version 5.0.1 (`npm i -D @vitest/coverage-v8@5.0.1`).

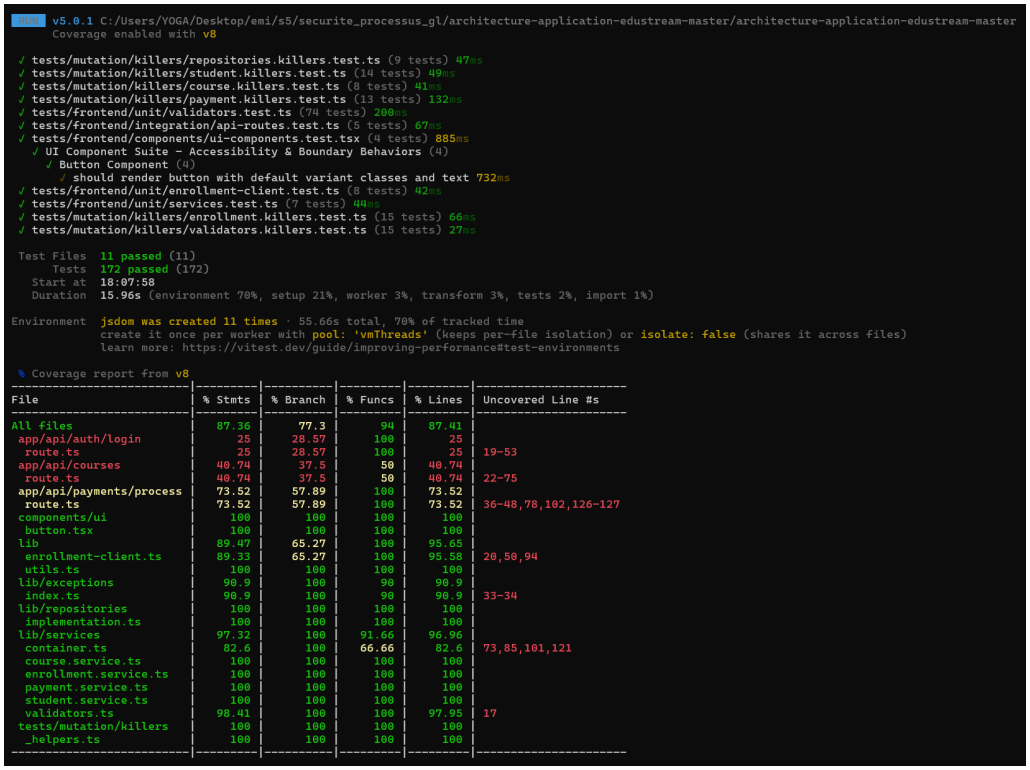


FIGURE 4 – Exécution réelle de `npm run test:frontend:coverage` : 172 tests passent sur 11 fichiers, avec le rapport de couverture

6.3 Couverture

Fichier	Instr.	Branches	Fonctions	Lignes
Ensemble du projet	**87,36 %**	**77,30 %**	**94 %**	**87,41 %**
`validators.ts`	98,41	100	100	97,95
`payment.service.ts`	100	100	100	100
`enrollment.service.ts`	100	100	100	100
`student.service.ts`	100	100	100	100
`course.service.ts`	100	100	100	100
`implementation.ts`	100	100	100	100
`container.ts` (hors campagne)	82,60	100	66,66	82,60
`exceptions`	90,90	100	90	90,90
route API `/auth/login`	25	28,57	100	25
route API `/courses`	40,74	37,50	50	40,74
route API `/payments/process`	73,52	57,89	100	73,52

TABLE 4 – Couverture V8 mesurée sur les 172 tests (valeurs en %)

La couverture des lignes passe de 75,81 % (rapport précédent) à **87,41 %**. Les six fichiers visés par la campagne de mutation sont couverts à 100 % (sauf la ligne 17 de `validators.ts`). Les zones faibles sont des routes d'API (`login`, `courses`, `payments/process`) : elles ne sont ni couvertes par les tests unitaires ni incluses dans la campagne.

6.4 Couverture et mutation ne mesurent pas la même chose

La couverture dit quelles lignes **s'exécutent**, le score de mutation dit si les erreurs sont **détectées**. Les deux se complètent :

— avant la campagne, `payment.service.ts` obtenait seulement 8 % au test de mutation : exécuter une ligne ne prouve pas qu'on la vérifie ;
— après les tests tueurs, les mêmes fichiers combinent 100 % de couverture et un score de mutation de 100 % (hors équivalents).

7 Synthèse

Fichier	Mutants (tour 1)	Tour 1	Tour 2	Tour 3
`validators.ts`	134	95,5 % (6 surv.)	98,5 % (2 surv.)	100,0 % (2 équiv.)
`payment.service.ts`	25	8,0 % (23 surv.)	100,0 % (0 surv.)	100,0 % (0 équiv.)
`enrollment.service.ts`	21	71,4 % (6 surv.)	100,0 % (0 surv.)	100,0 % (0 équiv.)
`student.service.ts`	28	39,3 % (17 surv.)	96,4 % (1 surv.)	100,0 % (0 équiv.)
`course.service.ts`	12	16,7 % (10 surv.)	91,7 % (1 surv.)	100,0 % (1 équiv.)
`implementation.ts`	47	46,8 % (25 surv.)	100,0 % (0 surv.)	100,0 % (0 équiv.)
Total	**267**	**67,4 %** (87 surv.)	**98,4 %** (4 surv.)	**100,0 %** (3 équiv.)

TABLE 5 – Score de mutation par fichier à chaque tour

Critère	Avant (tour 1)	Après (tour 3)
Score de mutation	67,4 %	**100,0 %**
Tests unitaires du service (hors Selenium)	81	**172** (+74 tests tueurs)
Mutants survivants	87	**0** (+ 3 équivalents justifiés)
Défauts réels corrigés	0	**2** (cartes valides refusées par la passerelle, nouvelle tentative impossible)
Code mort supprimé	0	1 (`minBalance`) + règles de refus arbitraires

TABLE 6 – Effet de la campagne de mutation

7.1 Ce que le test de mutation a apporté en plus des tests unitaires

— Il a montré que les tests passaient sans protéger plusieurs fonctionnalités annoncées : décrément des places, authentification, passerelle de paiement.
— Il a conduit à écrire des tests qui ont révélé **deux défauts réels** qu'aucun test précédent ne voyait.
— Il a donné un indicateur objectif (le score) pour décider quand les tests suffisent, ce que la couverture de lignes ne permet pas.

7.2 Limites de la démarche

— **Les mutants équivalents demandent un jugement humain** : décider qu'un mutant est équivalent suppose de bien comprendre le code (les quatre cas ci-dessus ont été vérifiés un par un).
— **Moteur maison** : il génère des mutants par transformation de texte et non via l'arbre syntaxique ; un outil professionnel comme Stryker produit davantage de variantes et gère les mutants invalides plus finement.
— **Périmètre** : seuls six fichiers et les tests unitaires sont concernés ; les routes API, les composants React et les tests Selenium ne sont pas mutés.
— **Ne rien lancer d'autre pendant une campagne** : le moteur modifie les sources en place (avec restauration garantie). Lors d'une exécution réelle, un `vitest run -coverage` lancé en parallèle a vu des mutants et a affiché de faux échecs (motif « Carte bloquée » vide, places non décrémentées). Les mêmes tests passent une fois la campagne terminée : c'était une erreur de manipulation de notre part, pas un défaut du code.
— **Un score élevé n'est pas une preuve** : il garantit que les erreurs simulées sont détectées, pas que le comportement attendu est le bon.

7.3 Reproductibilité

Les scores de mutation des tours 1 à 3 ont été calculés par le moteur décrit en section 2, avec l'exécuteur de tests intégré à Node.js . Les tests eux-mêmes ont ensuite été exécutés avec **Vitest** sur le poste du projet (section « Tests ajoutés, résultats et couverture »). La commande `npm run test:mutation` relance la campagne sous Vitest ; elle dure de 5 à 10 minutes.

8 Conclusion

Les tests de mutation ont fait passer la confiance accordée aux tests de « ils passent » à « ils détectent réellement les erreurs ». Partant d'un score de 67,4 % (alors que la couverture de lignes annonçait 75,81 %), la campagne a révélé que des fonctions critiques (paiement, décrément des places, authentification) n'étaient pas protégées. Les tests tueurs écrits pour combler ces trous ont mis au jour deux défauts réels : des **refus de cartes valides par la passerelle** (carte « bloquée », solde simulé) et l'**impossibilité de réessayer après un paiement refusé**, qui aggravait directement le défaut « vibe coded » des 10% de refus aléatoires. Après corrections, une carte qui passe Luhn est toujours acceptée, le score atteint **100,0 %** avec trois mutants équivalents expliqués, et la suite comprend 172 tests unitaires.

9 Benchmark : Stryker Mutator Face au Moteur Maison

Jusqu'ici, nous avons utilisé notre propre moteur de mutation. Pour vérifier qu'il est fiable, nous l'avons comparé à **Stryker Mutator**, l'outil le plus utilisé pour JavaScript et TypeScript, sur les mêmes six fichiers et avec les mêmes tests. Les sections 1 à 8 ne changent pas : leurs résultats viennent du moteur maison.

9.1 Pourquoi comparer

— **Contrôler notre moteur** : un score de 100 % n'a de valeur que si les mutants sont assez variés. Un autre outil permet de le vérifier.
— **Chercher d'autres trous** : Stryker fabrique des mutants que notre moteur ne produit pas (blocs vidés, tableaux, objets, chaînes vides). Un survivant que nous n'avions pas vu est un test manquant, que nous traitons comme avant : un test tueur écrit d'abord sur le code d'origine, une correction si le test échoue, puis une nouvelle campagne.
— **Comparer le temps** d'exécution d'une campagne.

9.2 Mise en place

Nous avons ajouté deux paquets de développement. Stryker utilise le Vitest déjà installé, et comme le projet utilise pnpm, l'installation se fait avec `pnpm` :

```
pnpm add -D @stryker-mutator/core @stryker-mutator/vitest-runner
```

Le fichier `stryker.config.json` vise exactement les mêmes fichiers que la campagne précédente :

```
{
  "testRunner": "vitest",
  "plugins": ["@stryker-mutator/vitest-runner"],
  "vitest": { "configFile": "vitest.config.mts" },
  "mutate": [ "validators.ts", "payment.service.ts", "enrollment.service.ts",
              "student.service.ts", "course.service.ts", "implementation.ts" ],
  "coverageAnalysis": "perTest",
  "reporters": ["clear-text", "progress", "html", "json"],
  "timeoutMS": 10000
}
```

Nous avons ajouté trois commandes : `npm run test:stryker` (Stryker seul), `npm run bench:mutation` (benchmark complet) et `npm run bench:report` (comparaison des résultats). Le benchmark lance d'abord le moteur maison puis Stryker, mesure la durée de chacun et produit les tableaux de la section « Résultats mesurés ».

9.3 Deux conceptions différentes

Aspect	Moteur maison	Stryker
Génération des mutants	Transformation du texte, avec classement des caractères (code, chaîne, commentaire, regex)	Analyse syntaxique : le code est lu comme un arbre, les mutations portent sur des nœuds
Exécution	Un mutant à la fois : le fichier est réécrit et Vitest est relancé	Code instrumenté une seule fois ; chaque mutant est activé par un interrupteur, sans réécrire les sources
Choix des tests	Tous les tests du fichier visé	Seuls les tests qui couvrent la ligne mutée (analyse de couverture par test)
Mutants invalides	Écartés via la transpilation TypeScript	Écartés par un vérificateur de types optionnel
Mutants équivalents	Liste `equivalents.json` justifiée à la main	Ignorés par commentaire ou option, ou laissés survivants
Mutateurs	Douze opérateurs, dont les nombres (`n` → `n+1`)	Famille plus large (blocs, tableaux, objets, regex), sans opérateur sur les nombres
Rapports	Console et JSON	Console, JSON et page HTML interactive avec le code surligné
Sûreté des sources	Fichier muté puis restauré (`.mutation.bak`)	Copie de travail dans un dossier temporaire : les sources ne bougent pas

TABLE 7 – Comparaison des deux approches

9.4 Protocole du benchmark

Pour que la comparaison soit correcte, nous avons respecté les points suivants :

— mêmes fichiers, mêmes tests et même machine, avec des exécutions l'une après l'autre et aucune autre commande de test en parallèle (voir les limites de la démarche, section 7) ;

— le moteur maison est lancé **sans** la liste d'équivalents (`-no-equivalents`), car Stryker ne la connaît pas, et avec les 74 tests tueurs, y compris celui du tour 3. Il compte donc 3 survivants, contre 4 au tour 2 de la section 4 ;

— le score de Stryker est $\frac{\text{tués}}{\text{tués+survivants+sans couverture}}$. Un mutant « sans couverture », qu'aucun test n'exécute, compte comme survivant ; le moteur maison n'a pas cette catégorie ;

— la durée est le temps réel de chaque outil. Comme le nombre de mutants n'est pas le même, nous comparons aussi le temps moyen par mutant ;

— les survivants sont comparés par ligne de code, car un même manque de test peut être vu par des mutants différents.

9.5 Résultats mesurés

	Moteur maison				Stryker			
Fichier	Mut.	Tués	Surv.	Score	Mut.	Tués	Surv.	Score
`validators.ts`	134	132	2	98,5 %	171	167	4	97,7 %
`payment.service.ts`	9	9	0	100,0 %	21	21	0	100,0 %
`enrollment.service.ts`	25	25	0	100,0 %	41	41	0	100,0 %
`student.service.ts`	28	28	0	100,0 %	51	51	0	100,0 %
`course.service.ts`	12	11	1	91,7 %	27	25	2	92,6 %
`implementation.ts`	47	47	0	100,0 %	104	104	0	100,0 %
Total	**255**	252	3	**98,8 %**	**415**	409	6	**98,6 %**

Table 8 – Résultats mesurés sur les mêmes six fichiers et les mêmes tests (les survivants de Stryker incluent les mutants ń sans couverture ż)

Mesure	**Moteur maison**	**Stryker**
Durée totale	1378 s	964 s
Mutants générés	255	415
Temps moyen par mutant	5,40 s	2,32 s

Table 9 – Performance (mesure de l'horloge murale, exécutions séquentielles)

Lignes de code où au moins un mutant survit	**Nombre**
Détectées par les deux outils	1
Moteur maison seulement	2
Stryker seulement	4

Table 10 – Recouvrement des survivants (par ligne de code)

Fichier	**Ligne**	**Mutateur Stryker**	**Remplacement**
`validators.ts`	16	BlockStatement (sans couverture)	`{}`
`validators.ts`	84	StringLiteral	`"Stryker was here!"`
`validators.ts`	86	Regex	`/^(5[1-5]|2(22[1-9]|2[`
`validators.ts`	92	StringLiteral	`"Stryker was here!"`
`course.service.ts`	50	CallExpression	`;`
`course.service.ts`	50	ConditionalExpression	`false`

Table 11 – Survivants de Stryker

```
Ran 2.09 tests per mutant on average.
-----------------------|-----------------|----------|-----------|------------|----------|----------|
                       | % Mutation score |          |           |            |          |          |
File                   |  total | covered | # killed | # timeout | # survived | # no cov | # errors |
-----------------------|--------|---------|----------|-----------|------------|----------|----------|
All files              |  98.55 |   98.79 |      378 |        31 |          5 |        1 |        0 |
 repositories          | 100.00 |  100.00 |      104 |         0 |          0 |        0 |        0 |
  implementation.ts    | 100.00 |  100.00 |      104 |         0 |          0 |        0 |        0 |
 services              |  98.07 |   98.39 |      274 |        31 |          5 |        1 |        0 |
  course.service.ts    |  92.59 |   92.59 |       25 |         0 |          2 |        0 |        0 |
  enrollment.service.ts| 100.00 |  100.00 |       41 |         0 |          0 |        0 |        0 |
  payment.service.ts   | 100.00 |  100.00 |       16 |         5 |          0 |        0 |        0 |
  student.service.ts   | 100.00 |  100.00 |       50 |         1 |          0 |        0 |        0 |
  validators.ts        |  97.66 |   98.24 |      142 |        25 |          3 |        1 |        0 |
-----------------------|--------|---------|----------|-----------|------------|----------|----------|
  13:59:21 (18632) INFO JsonReporter Your report can be found at:
file:///C:/Users/YOGA/Desktop/stryker-bench/reports/mutation/mutation.json
  13:59:21 (18632) INFO HtmlReporter Your report can be found at:
file:///C:/Users/YOGA/Desktop/stryker-bench/reports/mutation/mutation.html
  13:59:21 (18632) INFO MutationTestExecutor Done in 15 minutes and 58 seconds.

  >>> Stryker Mutator termine en 964 s (code 0)
```

FIGURE 5 – Exécution réelle de Stryker : résumé « clear-text » par fichier

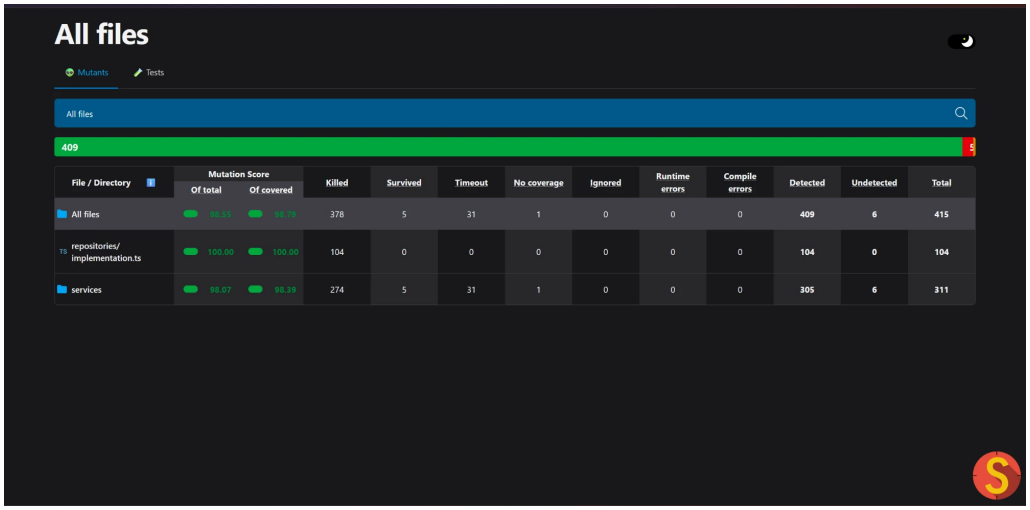


FIGURE 6 – Rapport HTML de Stryker : fichier `payment.service.ts` avec les mutants tués et survivants

```
> ========== BENCHMARK : moteur maison vs Stryker ==========
  Fichier                          Maison (mutants/tues/surv/score)   Stryker (mutants/tues/surv+nocov/score)
  lib/services/validators.ts       134/132/2/98.5%                    171/167/4/97.7%
  lib/services/payment.service.ts  9/9/0/100.0%                       21/21/0/100.0%
  lib/services/enrollment.service.ts 25/25/0/100.0%                   41/41/0/100.0%
  lib/services/student.service.ts  28/28/0/100.0%                     51/51/0/100.0%
  lib/services/course.service.ts   12/11/1/91.7%                      27/25/2/92.6%
  lib/repositories/implementation.ts 47/47/0/100.0%                   104/104/0/100.0%
  TOTAL                            255/252/3/98.8%                    415/409/6/98.6%
  Duree : maison 1378s, Stryker 964s
  Lignes survivantes : communes 1, maison seul 2, Stryker seul 4

> Ecrit : tests/mutation/reports/benchmark.json et benchmark_table.tex

  >>> Comparaison termine en 0 s (code 0)
```

FIGURE 7 – Sortie de `npm run bench:report` : comparaison des deux outils

9.6 Tableau comparatif

Critère	Moteur maison	Stryker 10.0.0
Mutants générés	255	415
Mutants tués	252	409 (dont 31 par délai dépassé)
Survivants	3	6 (dont 1 sans couverture)
Score de mutation	98,8 %	98,6 %
Fichiers à 100 %	4 sur 6	4 sur 6
Durée totale	1 378 s (≈ 23 min)	964 s (≈ 16 min)
Temps moyen par mutant	5,40 s	2,32 s
Tests rejoués par mutant	tous ceux du fichier visé	2,09 en moyenne (couverture par test)
Parallélisme	aucun (séquentiel)	7 processus
Mutations de nombres et de frontières	oui	non pour les nombres
Mutations de blocs, tableaux, objets, chaînes vides	non	oui
Mutants équivalents	liste justifiée à la main	aucune gestion automatique
Rapport	console et JSON	console, JSON et page HTML interactive
Dépendances	aucune	deux paquets, plugin lié à la version de Vitest
Version de Vitest requise	celle du projet	4.1.x (échec avec 5.0.5)

TABLE 12 – Comparaison des deux outils sur les mêmes six fichiers et les mêmes tests

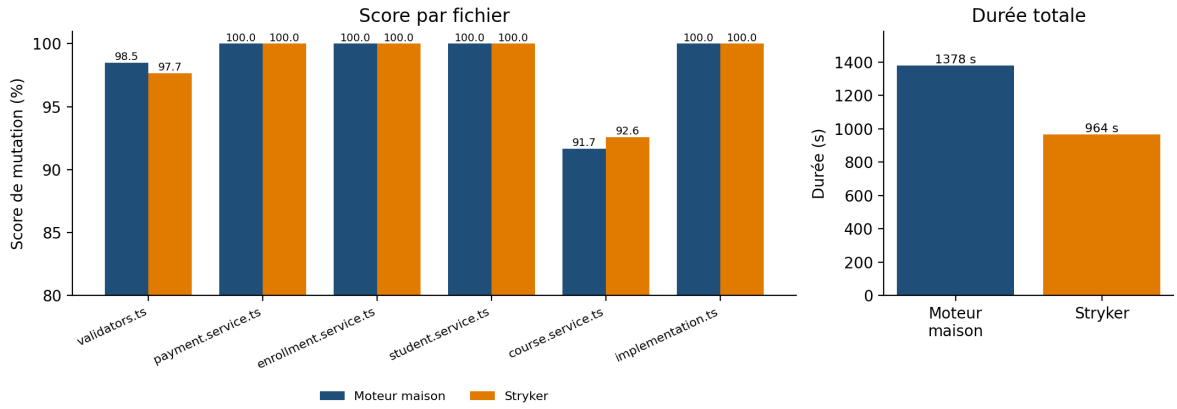


FIGURE 8 – Score de mutation par fichier et durée totale (l'axe des scores commence à 80 %)

9.7 Conditions réelles de l'exécution de Stryker

Nous avons exécuté Stryker 10.0.0 (plugin `@stryker-mutator/vitest-runner` 10.0.0, 7 processus en parallèle) sur les six fichiers. Les tableaux ci-dessus donnent ses résultats. Trois remarques sont nécessaires pour les lire.

1. **Version de Vitest.** Avec Vitest 5.0.5, la version du projet, Stryker ne détecte aucun mutant : sur `payment.service.ts`, nous avons obtenu 0 % (21 survivants

sur 21), ce qui contredit les tours 1 à 3. Le plugin est prévu pour Vitest 4.1.x. Nous avons donc lancé Stryker sur une copie du projet avec Vitest 4.1.10 (172 tests réussis). Avec cette version, le même fichier donne 21 mutants tués sur 21. Le moteur maison, lui, a tourné sur le projet d'origine, alors sous Vitest 5.

2. **Installation.** Il a fallu installer le paquet avec `pnpm`, car `npm i` échoue, et déclarer `"plugins": ["@stryker-mutator/vitest-runner"]` dans `stryker.config.json`. Sans cela, Stryker ne trouve pas le plugin. Le moteur maison n'a besoin d'aucune installation.
3. **Durées difficiles à comparer.** Stryker travaille sur 7 processus et ne rejoue que les tests qui passent par la ligne mutée (2,09 tests par mutant en moyenne). Le moteur maison travaille sur un seul processus et rejoue tous les tests du fichier. L'écart (964 s contre 1 378 s) vient donc autant de cette différence de conception que de la vitesse des outils.

9.8 Démarche d'analyse des écarts

Nous traitons chaque écart entre les deux outils comme une limite trouvée au tour précédent :

1. un survivant vu par les deux outils confirme une faiblesse connue, ou un équivalent déjà justifié dans `equivalents.json` ;
2. un survivant vu par Stryker seulement est soit un **test manquant** (nous écrivons un test tueur, d'abord sur le code d'origine), soit un **mutant équivalent** propre à Stryker (nous le justifions par écrit), soit un mutant d'un type que notre moteur ne produit pas (bloc, tableau ou objet vidé) ;
3. un survivant vu par le moteur maison seulement montre ce que Stryker ne regarde pas, par exemple les nombres.

9.9 Analyse des écarts

Les deux outils arrivent presque au même résultat : **98,8 %** pour le moteur maison (252 mutants tués sur 255) et **98,6 %** pour Stryker (409 sur 415, dont 31 arrêtés par dépassement de délai et comptés comme tués). Stryker produit 63 % de mutants en plus, mais n'a que 6 survivants. Dans les deux cas, les fichiers `payment`, `enrollment`, `student` et `implementation` atteignent 100 %. Les tests tueurs du tour 2 ne sont donc pas faits sur mesure pour notre moteur.

Survivant commun (1 ligne). `course.service.ts`, ligne 50 (`if (!course) throw new CourseNotFoundException`). Stryker y voit deux survivants : la condition remplacée par `false`, et la levée de l'exception supprimée. Nous avions déjà classé ce mutant comme équivalent au tour 3 : si le cours n'existe pas, `courseRepository.update` lève exactement la même exception. Stryker arrive à la même conclusion.

Survivants du moteur maison seulement (2 lignes). Il s'agit de `validators.ts`, ligne 23 (`+` devient `*` dans l'expression régulière) et ligne 36 (`>` devient `>=`). Ce sont deux de nos trois équivalents justifiés. Stryker ne produit pas la mutation de la ligne 23. À la ligne 36, il génère bien `digit >= 9`, mais le classe « délai dépassé » (*timeout*), donc tué, alors que ce mutant est équivalent. Cela vient sans doute du délai de 10 s avec 7

processus en parallèle, et non d'une vraie détection. Le score de Stryker est donc un peu trop favorable, et nous n'avons pas vérifié un par un ses 31 mutants en *timeout*.

Survivants de Stryker seulement (4 lignes).

- `validators.ts`, lignes 84 (`getCardType`) et 92 (`maskCardNumber`) : le remplacement de la chaîne vide par `"Stryker was here!"` dans `replace(/\D/g, "")` survit. Le résultat ne change que si le numéro **commence** (pour `getCardType`) ou **finit** (pour `maskCardNumber`) par un caractère qui n'est pas un chiffre, par exemple un espace. Aucun test ne couvre ce cas. C'est un **vrai manque de test**, que notre moteur ne pouvait pas voir car il ne modifie pas ce genre de chaîne vide.
- `validators.ts`, ligne 86 : l'expression `[3-6]\d{2}` devient `[3-6]\d`. Les deux versions ne diffèrent que pour un numéro de moins de quatre chiffres (par exemple `getCardType("235")`), un cas que la validation de longueur écarte avant. Un test direct le tuerait, mais dans le fonctionnement réel le mutant est presque équivalent.
- `validators.ts`, ligne 16 : le corps de `currentYear()` vidé est classé « sans couverture ». Nous n'avons pas cherché la cause ; elle vient probablement de la façon dont la date est fixée dans les tests.

Nous n'avons pas corrigé ces quatre points dans ce benchmark.

Bilan.

- Notre moteur est confirmé : son score (98,8 %) est très proche de celui de Stryker (98,6 %), et ses trois équivalents sont cohérents avec ce que Stryker observe.
- Stryker révèle **deux vrais manques de test** que nous n'avions pas vus : un numéro de carte avec un caractère qui n'est pas un chiffre au début ou à la fin.
- Notre moteur trouve des survivants que Stryker ne signale pas (lignes 23 et 36), mais ce sont des équivalents.
- Sur cette suite, Stryker est plus rapide (2,32 s contre 5,40 s par mutant) et fournit un rapport HTML pratique. En revanche, il impose une version précise de Vitest.

9.10 Limites du benchmark

- **Une seule machine, une seule exécution** : les durées varient avec la charge ; elles donnent un ordre de grandeur, pas une mesure statistique.
- **Mutants non comparables un à un** : les deux outils n'ont pas les mêmes mutateurs, donc des totaux différents ne signifient pas que l'un « trouve plus » d'erreurs.
- **Compatibilité des versions** : le plugin Vitest de Stryker 10.0.0 est validé avec Vitest 4.1.x ; avec Vitest 5.0.5, les mutants ne sont jamais activés et le score est nul (voir la sous-section « Conditions réelles de l'exécution de Stryker »).
- **Environnement jsdom** : la suite crée un environnement jsdom par fichier de test, ce qui pèse sur le temps de chaque mutant, quel que soit l'outil.